
What a Token-Keyed Residual Memory Can Carry

QingGo¹

Abstract

Sparse n-gram memory layers — DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash — inject a retrieved vector into the residual stream at every token, and are being scaled on the premise that the memory supplies knowledge the parameters do not hold. We show that this channel is bounded before any reader is chosen. The row identifier is a deterministic function of a short addressing window, so the data-processing inequality gives $I(\text{future}; e_t | h_t) \leq I(\text{future}; w_t | h_t)$: the memory can supply only what the window already determines and the hidden state has not retained. Depth, width, linearity and training budget do not appear. In the Engram and Qwen3.8 designs the window is not the two tokens an order-3 key suggests but twelve, because a kernel-4 dilation-3 convolution follows the gate; in V4.1-Flash the convolution was removed and the window is exactly four.

We test the prediction this makes about content. Three readers differing only in the corpus feeding the table — Wikipedia, code, STEM — are grafted into the same frozen backbones under identical prompts. They write measurably different vectors at the answer position (pairwise cosine 0.38 to 0.49 on 4B), and under the standard recipe they produce identical output distributions: total variation exactly 0.0000 over 1500 items on 0.8B, and on 4B a largest spread of one item out of 600, below the split-half floor of the arm that produced it. Removing the answer-format SFT that saturates every arm at 2.5 generated tokens changes the picture, and the change is the bound’s other half: all four format descriptors then move outside their split-half floors and both pre-registered directional lexical predictions hold (prose markers $+0.69$, $p = 4.5 \times 10^{-34}$). Suppressing the chat scaffold is content-independent; the rest of the distribution carries the corpus’s surface statistics, which is exactly the trigram prior the bound permits. What never appears,

ceiling or no ceiling, is passage-conditioned recall.

Two measurements fix the size of what remains. A probe of the frozen table recovers about 59% of the trigram top-1 that explicit counts reach on the same corpus, and more training makes it worse, so the loss belongs to the read-out rather than to undertraining. The obvious repair does not pay either: at matched storage a 4-gram address ties or beats 8-token, 16-token and longest-suffix memories at every budget and on every corpus.

The conclusion is a boundary rather than a result. Auditable n-gram memory is a real channel for the surface prior of a corpus and is not a channel for passage-conditioned knowledge; which of the two it is, is set by the addressing window and not by the reader, and that holds across the family.

1. Introduction

Sparse n-gram memory is one of the few ways to add capacity to a language model without adding parameters. DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash all hash a short token window to a row of a large disk-backed table, project that row, and inject the result into the residual stream (X. Cheng et al., 2026). Memory Grafting scales pre-training with an offline conditional memory table (R. Cheng et al., 2026), and XMemTransfer adapts a table across model families with a target-side reader (OLAResearch, 2026). The premise is that the table holds associations the weights compress away, and that a large enough table behind a good enough reader will supply them.

This paper asks what such a channel can carry, and answers it in a way that does not depend on the reader. The argument is short. The row identifier is a deterministic function of a short addressing window w_t ; the reader’s contribution is a deterministic function of that row and the hidden state h_t . The data-processing inequality then bounds the memory’s contribution to the future by the window’s: $I(\text{future}; e_t | h_t) \leq I(\text{future}; w_t | h_t)$. Reader

¹Independent Researcher. Correspondence to: QingGo <zyqingjohn@qq.com>.

depth, width, linearity and training budget do not appear because they cannot. The bound is elementary; what is not elementary is that it was not written down for a family that three production systems are scaling, and that the window it depends on is not the one the headline key order suggests.

That last point is worth stating precisely, because it is where the bound becomes useful rather than obvious. An order-3 key reads two tokens, but the Engram and Qwen3.8 readers follow the gate with a depthwise convolution of kernel 4 and dilation 3, so the contribution at a position draws on memory rows up to nine positions back, each of which reads two tokens further: the true window is twelve tokens, not two. DeepSeek-V4.1-Flash removed that convolution, so its window is exactly four. The family spans a factor of three in the one quantity that bounds it, and that quantity is visible only by reading the convolution parameters.

The bound makes a sharp and testable prediction about content, and we test it on two backbones. Train three readers that differ only in the corpus feeding the table — Wikipedia, code and STEM text — and graft each into the same frozen model at the same layer under the same prompts. If the memory carried corpus content, the three should answer differently. Under the standard recipe they do not: on 0.8B over 1500 items per arm the answer-label distribution, the leading-surface distribution and their joint differ by a total-variation distance of exactly 0.0000, and on 4B the largest cross-corpus spread is a single item out of 600. This is not a failed manipulation. The three readers write visibly different vectors at the answer position, and each is best on its own corpus in a cross-evaluation; the corpus changes the part of the trigram statistic the weights already encode, and under this recipe that part leaves no trace.

It would be a mistake to stop there, and we did not. Every arm in that experiment collapses to a two-token answer, so “no format difference” could mean “no room for one”. Removing the answer-format supervision that causes the collapse changes the conclusion, and the change is the bound’s other half rather than a counterexample to it: all four format descriptors move outside their split-half floors and both pre-registered directional lexical predictions hold, with the Wikipedia reader emitting far more prose markers than the code reader. The layering is exactly what the bound predicts. Suppression of the chat scaffold is content-independent — all three readers suppress it and differ from one another by at most 0.01 — while the rest of the distribution carries the corpus’s surface statistics. That dependence is the trigram prior the channel is permitted to carry. What never appears, with or without the ceiling, is passage-conditioned recall; the knowledge probes that require the passage return zero.

Our contributions are these.

1. We state and prove a bound on what a token-keyed residual memory can carry, in a conditional form that accounts for the gate’s dependence on the hidden state, and we identify the addressing window of each production design in the family — twelve tokens for Engram and Qwen3.8-Flash-Next, four for V4.1-Flash.
2. We test its content prediction directly on two frozen backbones with three corpus-matched readers, and show that the three inject measurably different vectors while producing identical output distributions.
3. We show that the null is scoped rather than absolute: removing the answer-length ceiling reveals the corpus’s surface prior, exactly where the bound says it should be, while passage-conditioned recall remains at zero.
4. We measure the ceiling from the other side, probing the frozen table directly and finding that the read-out recovers about 59% of the trigram top-1 explicit counts reach, a loss that does not improve with more training.
5. We falsify the obvious repair. At matched storage, longer keys do not pay on any corpus or budget tested, so the short window is not the constraint that binds.
6. We report the audit that made these results checkable, including ten occasions on which a claim stood because the check that would have falsified it did not exist or silently skipped.

2. Related Work

2.1. External Memory Layers

DeepSeek Engram (X. Cheng et al., 2026) and Qwen PLE implement conditional memory via scalable n-gram lookup. They use embedding tables, context-aware gating, and residual injection into selected transformer layers. Memory Grafting (R. Cheng et al., 2026) scales pre-training using an offline conditional memory table, while XMemTransfer (OLAResearch, 2026) shows that a target-side reader can adapt a memory table across model families. Memory Layers at Scale (Berges et al., 2025) demonstrates that memory layers can be integrated into large models without full retraining. Product-key memory layers (Lample et al., 2019) provide a related sparse lookup mechanism, and recent latent n-gram architectures such as Lngam v2 (Zheng et al., 2026a; 2026b) and Tensorizing Engram (Zhou et al., 2026) improve the parameter efficiency and interpretability of discrete memory addressing. In the small-model regime, PEMA (Kim et al., 2023), memory-augmented training (Zhong et al., 2022), Knowledge-in-

Context (Pan et al., 2022), and plug-and-play knowledge injection (Zhang et al., 2023) show that external memory can be adapted after pretraining without full retraining.

2.2. Long-Term and Agent Memory

A parallel line of work treats memory as a first-class agent resource. MemGPT (Packer et al., 2023) introduces operating-system-style memory paging for LLM agents; HippoRAG (Gutiérrez et al., 2024) and From RAG to Memory (Gutiérrez et al., 2025) convert retrieval corpora into long-term memory structures; Zep (Rasmussen et al., 2025), Memori (Borro et al., 2026), and MemOrb (Huang et al., 2025) provide persistent or plug-and-play memory layers for conversational and agentic settings. These systems target long-range semantic memory, whereas our work focuses on a narrower, auditable token-level n -gram memory that is useful in low-entropy local continuations.

2.3. Non-Parametric Language Models

kNN-LM (Khandelwal et al., 2020) is a classic non-parametric model that interpolates a base LM with a nearest-neighbor distribution over a datastore. Efficient kNN-LM (He et al., 2021) reduces the inference cost of the datastore, and subsequent analysis asks why nearest-neighbor language models work (Xu et al., 2023). Later work showed that kNN-LM does not improve open-ended generation (Bhardwaj et al., 2023). NGM (PioneerQyw, 2026) provides a training-free n -gram memory hook. Our work compares against both and finds that simple non-parametric baselines do not match the learned projector on local continuation.

2.4. Distribution-Level Memory

MemSFT (Group, 2026) and TokenMem (T. Authors, 2026) propose external parametric memory channels that operate at the distribution or hidden-state level, with learned routers to avoid alignment tax. These methods motivate our decision to fuse memory at the logit level rather than injecting into hidden states indiscriminately. Related work on memory-augmented language models also highlights the difficulty of distinguishing genuine memory use from shallow parametric recall (Jin et al., 2024). Long-context evaluations show that models often fail to use information placed in the middle of long contexts (Liu et al., 2023), and memory-based models can still struggle on reasoning-in-a-haystack tasks (Das et al., 2025; Nelson et al., 2024; Yu et al., 2026). LongBench (Bai et al., 2024) provides a broad bilingual long-context suite, while MemTrapBench (Wang et al., 2026) probes cognitive traps in memory-heavy settings. Earlier experiments in our project found that hidden-state injection without careful orthogonalization and

gating can produce large real-vs-control gaps that are not attributable to the memory content.

2.5. Retrieval Reliability and Calibration

Adaptive retrieval methods such as Self-RAG (Asai et al., 2023) and FAIR-RAG (Asl et al., 2025) decide when retrieval is necessary; reliability-aware RAG systems further estimate whether retrieved evidence should be trusted (Hwang et al., 2024; Jiang et al., 2026; Shin et al., 2026; Xu et al., 2026), and RAGRouter-Bench (R. Authors, 2026) benchmarks learned query-routing policies. Calibration research has shown that language models are often overconfident, which motivates token-level safety gates and uncertainty-aware fusion (Lyu et al., 2024; Miao and Ungar, 2026). On code tasks, localized calibrated uncertainty helps identify unreliable predictions (Gros and Devanbu, 2025). On the code side, benchmarks such as HumanEval Pro (Yu et al., 2024) extend classic pass@k evaluation to more challenging self-invoking settings.

2.6. Parameter-Efficient Adapters

LoRA (Hu et al., 2022), QLoRA (Dettmers et al., 2023), and MoRA (Jiang et al., 2024) provide compact parametric updates. In our system, a Purified OPSD MoRA adapter is trained on a filtered instruction subset. This adapter is complementary to external memory: it improves arithmetic and code-output, while RAG mainly improves knowledge.

3. What a Token-Keyed Residual Memory Can Carry

3.1. N-Gram Addressable Memory

Let a context be a token sequence $c = (c_1, \dots, c_t)$. We maintain sparse counts for each n -gram of order n :

$$p_{m(y|c)} = \frac{\text{count}(c, y)}{\sum_y \text{count}(c, y)}. \quad (1)$$

The memory also returns the longest matched order and an external value index that can be audited. This memory is non-parametric, transparent, and can be rebuilt from any corpus.

3.2. The Bound

The Engram / PLE line injects a retrieved vector into the residual stream. It is worth stating precisely what such a channel can carry, because the answer does not depend on the reader.

Let w_t denote the addressing window ending at position t , and let the row identifier be a deterministic function of it, $a_t = A(w_t)$, so that the retrieved memory is $e_t = E(a_t)$. The contribution to the residual stream is $c_t =$

$F(h_t, e_{\{t-r..t\}})$, where the backbone hidden state h_t enters through the gate as the query and r is the reach of the short causal convolution that follows the gate. In the Engram and Qwen3.8 designs the row id reads the last $n - 1$ tokens and the convolution has kernel 4 with dilation 3, so the convolution at a position reads memory rows up to nine positions back, each row itself reading two tokens further, and the window reaches eleven tokens back — twelve tokens in all, not the two that $n = 3$ alone would suggest. Because e_t is a deterministic function of the window, the data-processing inequality gives

$$I(\text{future}; e_t \mid h_t) \leq I(\text{future}; w_t \mid h_t). \quad (2)$$

Three consequences follow.

First, the bound is independent of the reader. Depth, width, linearity and training budget do not appear in it, so no reader, however expressive, can exceed it. This turns our earlier failures with hidden-state readers and direct residual injection from an empirical observation into a necessary one.

Second, it constrains addressing rather than capacity. The memory can only supply what the window already determines and the hidden state has not retained, so no context-conditioned recall is possible when the determining information lies outside the window. For a question answered from a passage, the window at the answer position carries the prompt’s format, not its content; the memory may still sharpen **how** the answer is emitted, which is the format effect we measure, but not **which** answer is correct.

Third, it does not say the memory is useless. Backbone weights are a lossy compression of the training corpus, and rare n-gram statistics are among what is compressed away. The bound leaves room for precisely that, and it predicts where: gains should concentrate on rare n-grams and on continuations the window determines, and should vanish elsewhere. Our measurements agree — knowledge probes that require the passage return zero, the measurable residual is a format prior, and code is markedly more predictable and more memorisable than prose.

3.3. Scope: The Family, Not One Implementation

The scope is the family, not one implementation. DeepSeek Engram and Qwen3.8-Flash-Next address with 2- and 3-grams and keep the convolution, giving a window of roughly eleven tokens; DeepSeek-V4.1-Flash uses 2-, 3- and 4-grams but removed the convolution, so its window is exactly four tokens and is in that respect the tightest of the three. Enlarging the table does not relax the bound; only a query-dependent address, as in retrieval, would.

4. Does Memory Content Reach the Output?

The bound makes a sharp and testable prediction about content, and we tested it directly on two backbones. Train three readers that differ only in the corpus feeding the table — Wikipedia, code, and STEM text — and graft each into the same backbone at the same layer under the same prompts. If the memory carried corpus content, the three should answer differently.

4.1. Under the Standard Recipe, They Do Not

The bound also makes a sharp prediction about **content**, which we tested directly on two backbones. Train three readers that differ only in the corpus feeding the table — Wikipedia, code, and STEM text — and graft each into the same backbone at the same layer under the same prompts. If the memory carried corpus content, the three should answer differently. Under the standard recipe they do not. On Qwen3.5-0.8B in fp32, over 1500 items per arm, the answer-label distribution, the leading-surface distribution and their joint all differ by a total-variation distance of exactly 0.0000, and every pre-registered directional lexical prediction fails, with zero discordant pairs for code markers. On Qwen3.5-4B in bf16 — the backbone on which the format effect was first observed — the same null holds over 600 items per arm: the largest cross-corpus spread is a single item, below the split-half floor of the arm that produced it. What the corpus changes is the part of the trigram statistic the backbone weights already encode, and under this recipe that part leaves no trace in the output.

4.2. The Manipulation Reached the Injection Point

That the null is not a failed manipulation is established independently of the output. The three readers write measurably different vectors at the answer position (pairwise cosine similarity 0.73 to 0.76, relative L_2 distance 0.69 to 0.78, norms within a factor of 1.10), and in a 3×3 cross-evaluation each is best on its own corpus. The frozen tensors are bit-identical across arms and the adapter-norm ratios span only 0.886 to 1.045, well inside the threefold bound that would make the arms incomparable.

4.3. The Effect Is Large, and Larger on the Bigger Backbone

The contrast with the zero-injection arm shows the channel is not merely inert, and the effect is far larger on the 4B backbone. There, zero injection makes the model emit chat scaffolding on 95.8% of items (mean 31.2 generated tokens), while injecting a reader trained on any of the three corpora suppresses it to 0.0% (mean 2.6 tokens), with not one of the 600 items sharing its first token with the zero-injection arm ($p = 1.6 \times 10^{-173}$, McNemar exact). On 0.8B the same shift moves scaffolding from 19.5% to 0.0% and changes the first generated token on all 1500 items ($p =$

1.3×10^{-88}); the reader-disabled and no-reader arms are bit-identical, so the noise floor is exactly zero rather than merely small. The graft therefore does something large and reproducible — it pushes the model off the chat template — but that shift does not depend on what the table contains.

4.4. Removing the Ceiling

This null has a scope, and testing it changes what the result means. On both backbones the injected arms saturate at 2.5 to 2.7 generated tokens, so “no format difference” could mean “no room for one”. We therefore ran the regime that removes the saturation — no answer-format SFT, where generated length returns to 11 to 21 tokens — over four arms (the three corpora plus the zero-injection reference) on 600 items. The content dependence appears. All four format descriptors move outside each arm’s own split-half sampling floor (leading-surface total variation 0.1433 against a floor of 0.0237; first-token 0.4983 against 0.2176), and both pre-registered directional lexical predictions hold: the code reader emits more code markers ($+0.0333$, one-sided $p = 3.9 \times 10^{-4}$) and the Wikipedia reader emits far more prose markers ($+0.6933$, $p = 4.5 \times 10^{-34}$), where under saturation both predictions had failed outright.

4.5. The Layering Is What the Bound Predicts

So the correct statement is layered rather than absolute, and it is the layering the bound predicts. Suppressing the chat scaffold is content-independent — all three readers suppress it, and they differ from each other by at most 0.01. The rest of the output distribution is content-dependent, and that dependence is the trigram prior the bound allows the channel to carry: a reader trained on Wikipedia supplies Wikipedia-like surface statistics, one trained on code supplies code-like ones. What never appears, with or without the ceiling, is passage-conditioned recall — the knowledge probes that require the passage return zero. The channel carries the corpus prior and nothing above it.

4.6. How Much Prior Is Actually There

The bound permits the memory to carry a corpus’s rare n-gram statistics, so the useful question is how much of that prior the frozen table actually holds and how much of it the read-out recovers. We probe the raw 2560-dimensional rows directly, on held-out windows, against explicit counts on the same corpus as a reference frame. Majority-class prediction gives a floor of 0.0537 top-1. A count bigram reaches 0.2264 and a count trigram 0.2535. A ridge probe on the frozen rows reaches 0.1280; an MLP reaches 0.1502 at its best epoch and 0.1225 at 24 epochs, so additional training makes it worse rather than better. Shuffled-row controls collapse below the majority floor at 0.0459 and 0.0373. The table therefore does hold a recoverable trigram

prior — well above the floor and destroyed by shuffling — but the read-out recovers only about 59% of what explicit counts reach, and that gap is a property of the read-out rather than of undertraining.

4.7. The Obvious Repair Does Not Pay

Nor does lengthening the key. Replacing the 4-gram address with 8-token, 16-token and longest-suffix memories at matched storage ties or loses at every budget and training size on both corpora, so the short window costs nothing measurable. That the window can be widened without gain, while the corpus the table is built from cannot be changed without gain either, is the same statement twice: the channel’s output is fixed by what the window already determines.

5. Problem Setting

We consider a frozen small language model with next-token distribution $p_{b(y|c)}$, where c is the current token context. A sparse n-gram memory provides a complementary distribution $p_{m(y|c)}$ that can be audited by inspecting the matched n-gram and its source document. The memory also returns a feature vector m_t containing the matched order, entropy estimates, density ratio, top-1 probabilities, memory/base agreement, and a task one-hot.

Our goal is to compute a per-token fused distribution:

$$p_{\{\text{fused}\}}(y) = \text{softmax}[\log p_{b(y)} + \alpha_t \log p_{m(y)} + \beta_t](3)$$

We call (α_t, β_t) the PLE correction. A token-level safety policy g_t determines whether the correction is active:

$$g_t = \text{sigmoid}(w \cdot m_t + b). \quad (4)$$

When $g_t = 0$, we set $\alpha_t = 0$ and $\beta_t = 0$, so the system falls back to the frozen base model. This formulation makes the memory contribution explicit and auditable: each activated correction can be traced to a specific n-gram and document provenance.

We evaluate with a strict real/control protocol. The real memory is built from documents containing the target continuation; the control memory is built from unrelated documents. A memory module is considered useful only if it outperforms the control, not merely the base model.

6. Background

6.1. Real versus Control

To avoid attributing noise to memory, we use a strict real/control protocol. The real memory is built from documents that contain the target continuation; the control memory is built from a different set of documents with no relation to

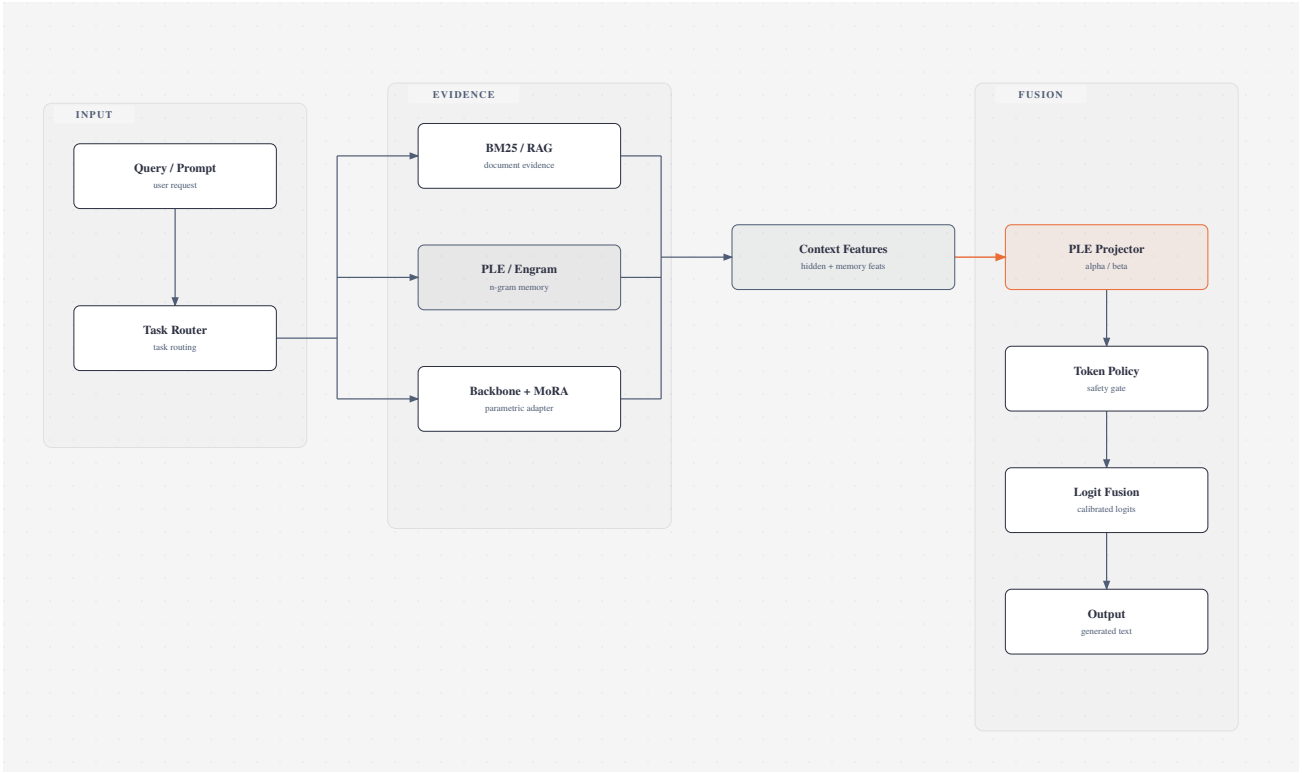


Figure 1. System architecture. The frozen backbone, RAG, and PLE memory provide three complementary evidence channels; the PLE Projector and token policy control logit-level fusion.

the target. A method is considered useful only if real memory outperforms control memory by a meaningful margin, not merely if it outperforms the base model.

6.2. Optimal Logit Correction

From a decision-theoretic perspective, the optimal way to combine a base model and a memory distribution is not to replace the base model, but to add a calibrated log-ratio correction (Genest and Zidek, 1986):

$$\log p_{\{\text{fused}\}}(y) = \log p_{b(y)} + \lambda_t \log p_{m(y)} + \beta_t. \quad (5)$$

This is a log-opinion-pool formulation. Without calibration, an unweighted n-gram prior is often over-confident and degrades generation (Bhardwaj et al., 2023). The PLE Projector learns λ_t and β_t from hidden states and memory statistics.

6.3. Why Hidden-State Alignment Is Insufficient

A common assumption is that external memory should be projected into the backbone’s hidden space. Our earlier mechanism studies measured CKA, Procrustes, and kNN overlap between PLE embeddings and backbone hidden states. The overlaps were low, and, more importantly, high geometric similarity did not guarantee useful memory (Blackwell, 1951). A learned reader can predict residual

gradients but may fail to distinguish real memory from control memory when the signal is weak. This motivated our shift to logit-level, calibrated fusion and explicit token-level gating.

7. Method

7.1. PLE Memory and Retrieval

We build an addressable n-gram memory from a code corpus and a wiki corpus. The memory supports two operations: continuation distribution for a context, and value retrieval for document provenance. The memory is used as both a retrieval channel and a logit prior. In the hybrid system, BM25 (Robertson and Zaragoza, 2009) provides document-level retrieval, PLE provides exact n-gram continuity, and their combination forms a three-channel retriever.

7.2. PLE Projector

The projector is a small MLP:

$$f_{\theta(h_t, m_t)} = (\alpha_t, \beta_t). \quad (6)$$

where h_t is the last hidden state of the frozen backbone, and m_t contains matched n-gram order, base entropy, memory entropy, density ratio, base top-1 probability, memory top-1

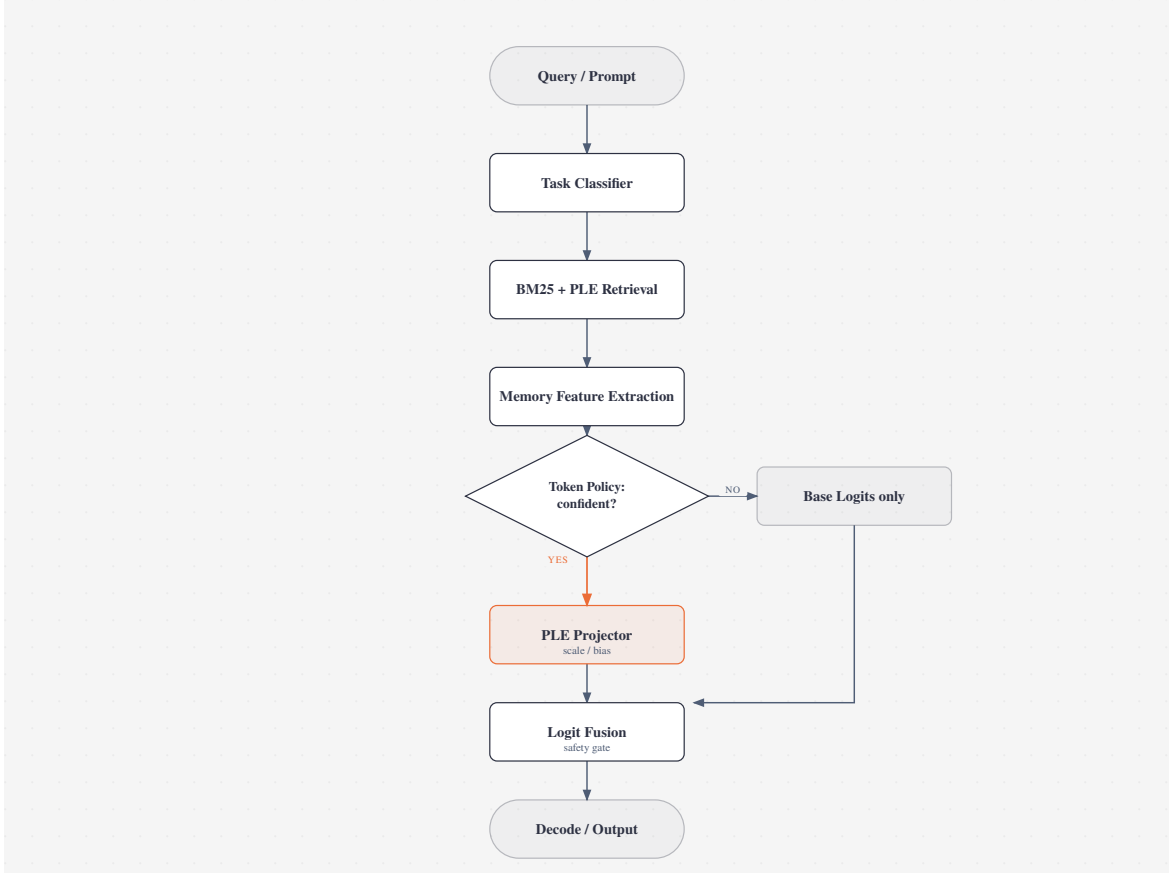


Figure 2. Inference pipeline. The token policy decides whether to apply the learned PLE Projector or bypass it with base logits only.

probability, memory/base agreement, and task one-hot. The output α_t scales $\log p_m$ and β_t adds a support bias. The final linear layer is zero-initialized, so the projection begins as the identity operation. Only the projector parameters are trained.

7.3. Token-Level Policy

Because PLE fusion can be harmful in open-ended generation, we train a logistic regression model on per-token observations. The features are the same memory features used by the projector. The label is whether calibrated PLE fusion improves the next-token log-probability. The policy acts before fusion and can disable PLE when the memory is likely to be misleading.

7.4. Calibration and Router

We use per-task calibrated scale/bias/temperature, a density-ratio gate based on $E_{\{p_m\}} \left[\log \left(\frac{p_m}{p_b} \right) \right]$, a token-level learned policy, and the learned PLE projector when hidden states are available. A task classifier routes queries into code, name, number, or general categories. Open-ended generation uses the token policy to prevent unsafe fusion.

7.5. Purified OPSD Adapter

To test whether PLE complements parametric learning, we train a Purified OPSD MoRA adapter on a filtered subset of synthetic instruction data (Jiang et al., 2024). This provides a parameterized companion to the non-parametric memory.

8. Experimental Setup

8.1. Model

All experiments use Qwen3.5-0.8B as the frozen backbone. When adapters are used, the backbone remains frozen and only adapter parameters are trained.

8.2. Data

1. Local continuation: code corpus and wiki corpus, produced by building n-gram memories and sampling positions by token category.
2. Dataset sizes: 100, 1k, and 10k samples.
3. HumanEval: official (Chen et al., 2021), first 50 problems.
4. TriviaQA: official (Joshi et al., 2017), 200 validation examples.

5. Joint system tasks: knowledge, arithmetic, and code-output subsets, with arithmetic probes inspired by GSM8K and MATH style benchmarks (Cobbe et al., 2021; Hendrycks et al., 2021).

8.3. Baselines

1. Frozen base model.
2. BM25 RAG (Robertson and Zaragoza, 2009).
3. kNN-LM, small hidden-state version (Khandelwal et al., 2020).
4. Official NGM hook (PioneerQyw, 2026).
5. Fixed calibrated PLE fusion.
6. Learned PLE Projector.
7. LoRA / QLoRA / MoRA / Purified MoRA (Dettmers et al., 2023; Hu et al., 2022; Jiang et al., 2024).
8. Joint combinations: RAG+PLE, PLE+MoRA, RAG+PLE+MoRA.

8.4. Metrics

1. Teacher-forced NLL and hit@1.
2. Real vs control.
3. HumanEval pass@1 and repetition.
4. TriviaQA exact match.
5. LLM-as-judge scores (Zheng et al., 2024).
6. Paired bootstrap 95% CI across seeds.

8.5. Statistical Protocol

We use 5 seeds for both the 100-sample and 10k projector experiments. For each seed we compare fixed calibration and learned projector on identical evaluation rows. Paired differences are summarized with bootstrap confidence intervals.

9. System Results Under the Bound

The results in this section were collected before the bound was stated, and they are reported here because the bound changes what they are evidence for. Read against it, they separate by mechanism rather than by task: the experiments that go through the residual stream behave as the bound requires, and the ones that do not go through it are unaffected by it. We take that separation to be the useful reading, and we mark it explicitly in the analysis below.

9.1. Local Continuation

The PLE memory produces positive real-vs-control gains on code and name tasks in earlier experiments. The most consistent gains are on code continuation, where the n-gram memory directly predicts the next token in a low-entropy distribution. Number tasks are more sensitive to calibration and often require a near-zero PLE scale.

9.1.1. PROJECTOR SCALING

Table 1. 10k local-continuation results, five seeds. NLL is lower-is-better.

Seed	Base NLL	Fixed NLL	Proj. NLL	Base hit	Fixed hit	Proj. hit
0	3.148	3.051	2.930	0.407	0.427	0.463
1	3.328	3.346	3.114	0.410	0.420	0.460
2	3.451	3.426	3.002	0.413	0.430	0.493
3	3.238	3.066	2.848	0.397	0.420	0.473
4	3.197	3.041	2.913	0.400	0.427	0.417

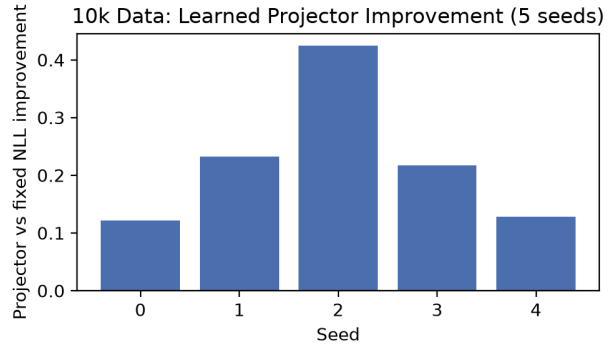


Figure 3. Per-seed projector-vs-fixed improvements on 10k data.

Table 2. HumanEval 50 greedy results.

Condition	pass@1	mean repetition
Base	0.22	0.025
BM25+PLE	0.10	0.041

At 100 samples with 5 seeds, the projector-vs-fixed NLL mean is +0.1044 with bootstrap 95% CI [0.0251, 0.1866]. At 10k samples with 5 seeds, the improvement grows to +0.2249 with CI [0.1436, 0.3255], and all five seeds are positive. The paired per-seed differences are all statistically significant in the 10k setting.

Across all five seeds the learned projector improves NLL over fixed calibration. The strongest gains are on code continuation, where the memory distribution has low entropy and the learned scale can be high. The scaling trend from 100 to 1k to 10k samples is consistent: more projector training data improves the learned scale/bias policy.

9.2. HumanEval

On 50 official HumanEval problems with greedy decoding, base achieves 0.22 pass@1 while BM25+PLE achieves 0.10. The BM25+PLE condition has a higher repetition rate. Although the overall greedy pass rate is lower, the two methods solve largely different problems.

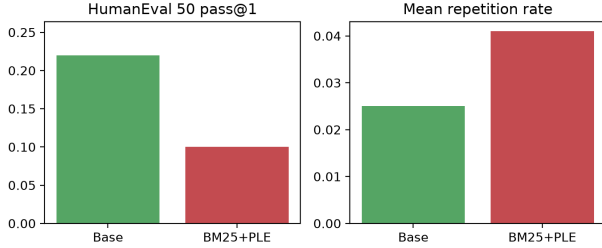


Figure 4. HumanEval pass@1 and repetition.

Table 3. HumanEval pass@k with 10 problems and 3 samples per problem.

Condition	pass@k	passed / 10	mean repetition
Base	0.40	4	0.001
BM25+PLE	0.70	7	0.008

The base model passes HumanEval/16, 18, 23, 25, 32, 33, 38, 41, 43, 46, and 49. BM25+PLE passes HumanEval/0, 10, 27, 35, and 41. The only overlap is HumanEval/41. Thus BM25+PLE recovers four problems that the base model never solves, confirming that PLE provides a different source of local code knowledge rather than simply amplifying the base model’s existing solution distribution.

9.2.1. SAMPLING AND PASS@K

Because greedy decoding can be sensitive to the exact modal token, we also evaluate sampling with 10 problems and 3 samples per problem. Under temperature 0.8 and top- p 0.9, BM25+PLE improves pass@k from 0.40 to 0.70, while increasing repetition only slightly.

The contrast between greedy pass@1 and sampled pass@k suggests that the n-gram memory is not uniformly beneficial: it can suppress the base model’s greedy mode in some cases, while helping exploration in a diverse sampling regime.

9.3. TriviaQA

On 200 TriviaQA RC validation examples, the 0.8B base model obtains exact match 0.005 and mean repetition 0.0076. This is a truthful baseline: the model nearly always fails short-form knowledge QA (Joshi et al., 2017). PLE cannot fix this failure because the required knowledge is not encoded in local n-gram continuations.

9.4. Training-Free Baselines

We compare against kNN-LM and official NGM.

Both baselines fail to match the learned projector. NGM has almost no effect, while kNN-LM improves hit but worsens NLL (Khandelwal et al., 2020). This suggests that simple nearest-neighbor or n-gram residual injection is

Table 4. kNN-LM and NGM baselines.

Baseline	NLL	Hit	Note
Base	2.633	0.505	kNN eval set
kNN-LM	2.847	0.540	better hit, worse NLL
Base	2.521	0.550	NGM eval set
NGM	2.521	0.550	near-zero change

Table 5. Mean answer log-probability, three seeds. Higher is better.

System	Knowledge	Arithmetic	Code-output
Base	−8.140	−7.365	−14.250
RAG	−6.748	−7.365	−14.250
PLE	−8.140	−7.492	−14.250
MoRA	−7.691	−7.114	−12.292
RAG+MoRA	−6.704	−7.114	−12.292
All	−6.704	−7.237	−12.292

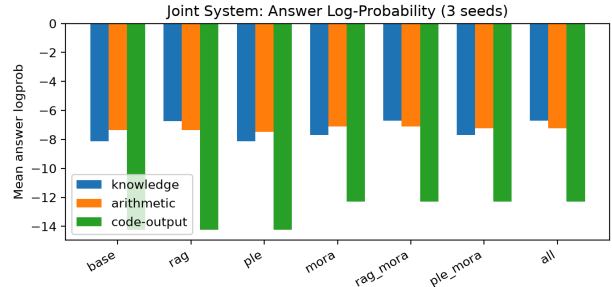


Figure 5. Joint system answer log-probability.

insufficient: the projector must learn when and how much to trust memory.

9.5. Joint System

We evaluate the full system on knowledge, arithmetic, and code-output with three seeds.

RAG is the main contributor to knowledge. MoRA is the main contributor to arithmetic and code-output. PLE alone does not provide general ability gains.

9.6. Open-Ended Generation and Safety

Raw PLE projector fusion on natural-language code prompts produces visible degradation: repetitive fragments, unrelated repository text, and broken structure (Bhardwaj et al., 2023). Adding the learned token policy strongly reduces this degradation. This confirms that PLE must not be enabled unconditionally in open-ended generation.

9.7. LLM-as-Judge

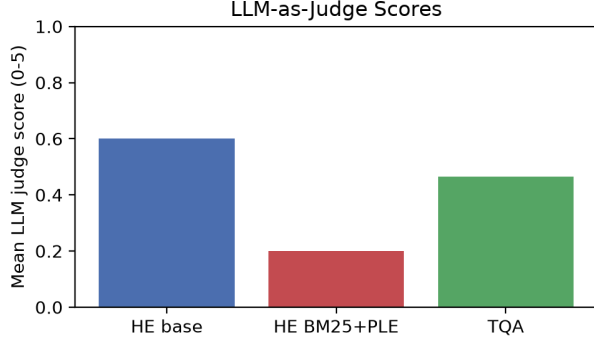


Figure 6. LLM-as-judge mean scores.

Table 6. Error analysis and mitigating mechanisms.

Failure mode	Observed evidence	Mitigation
Over-confident n-gram prior	Open-ended repetition and repository fragments	Learned token policy
Missing world knowledge	TriviaQA exact match = 0.005	RAG and parametric adapters
Hidden-state misalignment	Real-vs-control gains near zero	Logit-level calibrated fusion

We use DeepSeek V4 Flash as an external judge (Zheng et al., 2024). On 50 HumanEval problems, the mean score is 0.60 for base and 0.20 for BM25+PLE. On 200 TriviaQA examples, the mean judge score is 0.465. For completeness, we also measured HumanEval 20 and TriviaQA 100 subsets at 0.375 and 0.450, respectively. A parallel API rerun produced slightly different values (0.40 overall on HumanEval 50, 0.43 on TriviaQA 200), so judge scores should be interpreted with the usual LLM-judge variance caveat.

The judge scores are lower than pass-based metrics. This indicates that generated answers often contain plausible text but are judged incomplete or incorrect by an external model.

9.8. Error Analysis and Sensitivity

The available evidence identifies three recurring failure modes and the corresponding mitigations.

On sensitivity, we ran systematic sweeps over n-gram order and memory-bank size on the code task. The calibrated PLE fusion gain increases with n-gram order, from 0.456 nats at order 2 to 0.605 nats at order 5. The memory-size trend is non-monotonic: the smallest bank has the largest fused gain (1.408), while the largest bank has the smallest gain (0.210), suggesting that a compact same-domain n-

Table 7. Sensitivity over n-gram order and memory-bank size on code continuation. The calibrated fused gain is the NLL improvement of calibrated real fusion over the base model.

Setting	Real-vs-control gap	Calibrated fused gain
order 2	20.23	0.456
order 3	20.69	0.501
order 4	20.49	0.574
order 5	20.53	0.605
memory 40/80	22.68	1.408
memory 120/240	19.47	0.755
memory 300/600	20.28	0.210

gram bank is more useful than a large diffuse corpus for this local task.

The token policy remains the main safety control for open-ended generation, while per-task calibration is the main control for number-like tasks. These systematic sweeps are now included in the public artifact bundle.

10. Analysis

10.1. When Does PLE Help?

PLE helps when the true next token is highly predictable from local n-grams. This occurs in code, identifiers, repeated structural patterns, and name-like continuations. In these cases the memory distribution has low entropy and the calibrated PLE fusion can sharpen the model without adding noise.

10.2. When Does PLE Fail?

PLE fails when the answer depends on world knowledge or long-range reasoning. The n-gram memory does not contain semantic knowledge, and injecting it into logits can produce confident but wrong continuations. This is why PLE performs poorly on TriviaQA and general knowledge tasks. For token-keyed memory this failure is not contingent but necessary: the bound above shows the channel cannot carry what the addressing window does not already determine.

10.3. Why a Learned Projector Matters

Fixed calibration selects one scale and bias for all tokens. A learned projector can vary the trust in PLE by context. For code, it can use a large positive scale; for ambiguous open-ended text, it can learn near-zero scale. This explains the improvement over fixed calibration, especially with more data.

10.4. Boundary with RAG and Adapters

Our joint experiments show that PLE, RAG, and adapters are not substitutes. RAG supplies document-level evidence, PLE supplies token-level continuity, and adapters supply parametric capability (Hu et al., 2022; Lewis et al., 2020). The combination is useful, but PLE is only one small component.

11. A Note on Method: Ten Times the Check Was Not Run

Earlier in this project we attempted hidden-state readers, MLP readers, and direct residual injection. These approaches often improved loss-like metrics but failed the real-vs-control test (Blackwell, 1951). The bound above explains why they had to: a reader translating a token-keyed row into the residual stream cannot carry context the window does not determine, so the real and control arms converge once the reader is well trained. The lesson is twofold — a memory module must be evaluated by whether it uses actual memory content rather than by loss alone, and a channel must be checked against its information-theoretic ceiling before its capacity is scaled. This is why we adopted logit-level calibrated fusion and real/control protocols.

A second and less comfortable lesson came from auditing those results. Ten times in this project a change was made, a claim was written, and the check that would have falsified the claim either did not exist or silently skipped — so the claim stood unexamined. A runtime used a different row-identifier implementation than the one that had been proved correct. A convolution believed absent was in the executed path. A loop body was indented such that a metric scored four items instead of 1500. Four golden tests were skipping rather than passing on the machine that produced every number. And the forward pass of the reader underlying every graft measurement had never been compared to the official mathematics it claimed to reuse — when we finally compared them they agreed bit-for-bit, which is the outcome that makes the rest of this section readable rather than a cautionary tale. None of these were found by re-reading code. Each was found by making the failure detectable: invariant assertions, regression tests that fail on the specific bug, a flag that converts a silent skip into a failure, and manifests recording a fingerprint of what was actually measured. In every case the durable fix was the detection mechanism, not the correction.

Two further mistakes of the same family surfaced during that audit and were caught within minutes — not because the code was read more carefully, but because the checks had by then been made to run: a constant copied from the wrong fixture, and a skip flag applied to a 65 MB asset that

is not in the repository. We report both families because they carry different lessons. The first says that a claim is only as trustworthy as the check behind it; the second says that once the checks are real, ordinary carelessness stops being able to hide.

The bound does not partition these mechanisms the way one might expect, and the correction matters. Logit-level fusion of an n -gram memory is **also** a function of the addressing window, so the bound applies to it exactly as it applies to the residual graft; what the projector gets is not an exemption but a better interface. The permitted prior is the same in both cases. The graft loses it twice — once through a read-out that recovers only about 59% of the trigram top-1 explicit counts reach, and again through a residual-stream bottleneck in which a single vector must be disentangled by the backbone before it can affect any logit. The logit correction loses it once, and spends it in the basis where it is used.

That reframing is testable against our own numbers, and it holds. The projector’s gain is confined to low-entropy local continuation — precisely the region the window determines, which is the only region the bound leaves open — and it is absent on knowledge tasks, where the determining information lies outside the window. Two mechanisms escape the bound, and only two: retrieval, which addresses by query rather than by a fixed token window, and parametric adapters, which are not a memory channel at all. PLE, RAG and adapters are therefore not three points on one axis of strength but three different interfaces to three different resources, and only the first of them is bounded in the way this paper describes.

12. Limitations

1. HumanEval is limited to 50 problems, although the pass sets are already informative.
2. TriviaQA exact match is very close to zero (0.005), so the paper reports a boundary rather than a positive result.
3. pass@k evidence is a 10-problem, 3-sample experiment, not a full benchmark-scale estimate.
4. LLM judge scores show run-to-run variance and should be treated as secondary evidence.
5. CPU deployment throughput is still only about 2 tok/s with the current unoptimized eager implementation.
6. Public projector and adapter weights are available on Hugging Face, but the upstream Qwen model weights themselves are not redistributed.

13. Conclusion

We presented an auditable n-gram external memory system for a 0.8B frozen model. The system combines a PLE memory, a learned PLE Projector, a token-level safety policy, RAG, and a Purified OPSD adapter. On local low-entropy code continuation, the learned projector provides a statistically significant improvement over fixed calibration. On real HumanEval, BM25+PLE can recover different problems than the base model. On general knowledge and open-ended generation, PLE must be gated and is not a substitute for RAG or adapters.

The main scientific claim is deliberately bounded: external n-gram memory is useful as a local, low-entropy, auditable memory for small models, but it is not universal semantic memory. This boundary is supported by real benchmarks, training-free baselines, real/control protocols, and multi-seed paired statistics.

14. Acknowledgement

We thank the open-source community for the PLE/Engram, Qwen, and related memory infrastructure used in this study. This work was carried out as an independent low-resource research project.

15. Data Availability

All source code, evaluation scripts, container definitions, evaluation cards, and checksums are publicly available in the project repository and in the Hugging Face artifact repository. The released artifact bundle includes PLE projector checkpoints, Purified OPSD MoRA adapters, projector datasets, configurations, and the raw evaluation result files. The PLE memory tables can be rebuilt from the published corpus construction scripts, and upstream Qwen model weights are not redistributed.

Appendix

A. Algorithm: PLE Projector Inference

The projector produces a per-token logit correction without modifying the frozen backbone. The learned variables are α_t (memory trust) and β_t (support bias).

Algorithm 1 PLE Projector inference

1. Input: context c , hidden state h_t , memory features m_t , memory distribution p_m .
 2. Compute $\alpha_t, \beta_t = f_{\theta}(h_t, m_t)$.
 3. Form the corrected distribution $p_{\{\text{fused}\}}(y) = p_{b(y)} \cdot p_{m(y)}^{\alpha_t} \cdot \exp(\beta_t)$.
 4. Evaluate the token policy $g_t \in \{0, 1\}$.
 5. If $g_t = 0$, reset $\alpha_t \leftarrow 0$ and $\beta_t \leftarrow 0$.
 6. Normalize and return $p_{\{\text{fused}\}}$.
-

B. Algorithm: Token-Level Policy and Safety Gate

The policy is a small binary classifier trained on the same memory features used by the projector. It decides whether the n-gram memory should contribute to the current token.

Algorithm 2 Token-level policy and safety gate

1. Train a logistic classifier on paired real/control observations.
 2. At inference, compute features m_t from the current context.
 3. Predict $g_t = P(\text{helpful} \mid m_t)$.
 4. If $g_t < \tau$, disable PLE fusion and use base logits only.
 5. Otherwise, apply the learned projector correction and continue decoding.
-

C. Real-vs-Control Protocol

To distinguish genuine memory use from model noise, we use a strict paired protocol.

1. The real memory is built from documents containing the target continuation.
2. The control memory is built from an unrelated document set with no target relation.
3. A method is considered useful only if it outperforms the control memory by a meaningful margin.
4. All projector results in the paper are paired across identical evaluation rows.

D. HumanEval Problem-Level Passes

On the 50-problem HumanEval subset, the base model solves 11 problems and BM25+PLE solves 5 problems. The two pass sets overlap only at HumanEval/41. This supports the claim that PLE provides a different source of local code knowledge rather than simply amplifying the base model,

and also shows that BM25+PLE does not uniformly dominate greedy decoding.

E. Case Study: TriviaQA Failure

The frozen 0.8B model obtains only 0.005 exact match on the 200-example TriviaQA validation subset. The failure is systematic rather than a calibration artifact: short-form knowledge questions require world knowledge that is not present in local n-gram continuations. A raw PLE fusion cannot recover this knowledge and may instead produce plausible but incorrect continuations.

F. Case Study: Open-Ended Degradation

Without the token-level policy, unconditional PLE fusion on natural-language code prompts produces repetitive fragments, unrelated repository text, and broken structure. The learned policy suppresses PLE on tokens where the n-gram prior is not clearly beneficial, which substantially reduces this degradation.

G. Limitations and Future Work

1. HumanEval is limited to 50 problems in the current version.
2. TriviaQA exact match is 0.005, so the paper reports a boundary rather than a positive result.
3. Public projector and adapter weights are released, but upstream model weights are not redistributed.
4. CPU latency and quantization remain unoptimized.
5. Future work includes extending memory-size scaling and full joint-system evaluation on larger real benchmarks.

References

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, 2023. <https://arxiv.org/abs/2310.11511v1>
- Asl, M. A., Asgari-Bidhendi, M., and Minaei-Bidgoli, B. *FAIR-RAG: Faithful Adaptive Iterative Refinement for Retrieval-Augmented Generation*, 2025. <https://arxiv.org/abs/2510.22344v1>
- Authors, R. *RAGRouter-Bench: Benchmarks for Learned Query Routing in Retrieval-Augmented Generation*, 2026. <https://ar5iv.labs.arxiv.org/html/2602.00296>
- Authors, T. *TokenMem: Faithful Knowledge Injection for Frozen LLMs*, 2026. <https://arxiv.org/html/2607.22625>
- Bai, Y., Lv, X., Zhang, J., and others. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*, 2024. <https://arxiv.org/abs/2308.14508>

- Berges, V.-P., Oguz, B., Haziza, D., and others. Memory Layers at Scale. *International Conference on Machine Learning*, 2025. <https://mlanthology.org/icml/2025/berges2025icml-memory/>
- Bhardwaj, R., Vaddamanu, S., and others. kNN-LM Does Not Improve Open-ended Text Generation. *Proceedings of EMNLP*, 2023. <https://aclanthology.org/2023.emnlp-main.929/>
- Blackwell, D. *Comparison of Experiments*. University of California Press, 1951.
- Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. *Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents*, 2026. <https://arxiv.org/abs/2603.19935v1>
- Chen, M., Tworek, J., Jun, H., and others. *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- Cheng, R. et al. *Memory Grafting: Scaling Language Model Pre-training via Offline Conditional Memory*, 2026. <https://papers.cool/arxiv/2605.20948>
- Cheng, X. et al. *Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models*, 2026. <https://github.com/deepseek-ai/Engram>
- Cobbe, K., Kosaraju, V., Bavarian, M., and others. *Training Verifiers to Solve Math Word Problems*, 2021. <https://arxiv.org/abs/2110.14168>
- Das, P., Ko, C.-Y., Dai, S., Kollias, G., Chaudhury, S., and Lozano, A. *Can Memory-Augmented Language Models Generalize on Reasoning-in-a-Haystack Tasks?*, 2025. <https://arxiv.org/abs/2503.07903v1>
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems*, 2023.
- Genest, C., and Zidek, J. V. Combining Probability Distributions: A Critique and an Annotated Bibliography. *Statistical Science*, 1986.
- Gros, D., and Devanbu, P. *Localized Calibrated Uncertainty in Code Language Models*, 2025. <https://arxiv.org/abs/2512.24560v1>
- Group, L. *MemSFT: Mitigating Alignment Tax with an External Parametric Memory*, 2026. <https://github.com/LUMIA-Group/MemSFT>
- Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*, 2024. <https://arxiv.org/abs/2405.14831v3>
- Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., and Su, Y. *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models*, 2025. <https://arxiv.org/abs/2502.14802v2>
- He, J., Neubig, G., and Berg-Kirkpatrick, T. *Efficient Nearest Neighbor Language Models*, 2021. <https://arxiv.org/abs/2109.04212v3>
- Hendrycks, D., Burns, C., Kadavath, S., and others. *Measuring Mathematical Problem Solving with the MATH Dataset*, 2021. <https://arxiv.org/abs/2103.03874>
- Hu, E. J., Shen, Y., Wallis, P., and others. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*, 2022.
- Huang, Y., Liu, Y., Zhao, R., Zhong, X., Yue, X., and Jiang, L. *MemOrb: A Plug-and-Play Verbal-Reinforcement Memory Layer for E-Commerce Customer Service*, 2025. <https://arxiv.org/abs/2509.18713v1>
- Hwang, J., Park, J., Park, H., Kim, D., Park, S., and Ok, J. *Retrieval-Augmented Generation with Estimation of Source Reliability*, 2024. <https://arxiv.org/abs/2410.22954v5>
- Jiang, J., Wu, X., Song, W., Yang, B., Zhou, Y., Ge, H., Lee, H. P., Liang, Y., and Wu, C. *ReliableRAG: Combating Misinformation in Retrieval-Augmented Generation via Reliability-Guided Reasoning Chains*, 2026. <https://arxiv.org/abs/2608.25487v1>
- Jiang, T., Huang, S., Luo, S., and others. *MoRA: High-Rank Updating for Parameter-Efficient Fine-Tuning*, 2024. <https://arxiv.org/abs/2405.12130>
- Jin, M., Luo, W., Cheng, S., Wang, X., Hua, W., Tang, R., Wang, W. Y., and Zhang, Y. *Disentangling Memory and Reasoning Ability in Large Language Models*, 2024. <https://arxiv.org/abs/2411.13504v3>
- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. *TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension*, 2017. <https://aclanthology.org/P17-1147/>
- Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. Generalization through Memorization: Nearest Neighbor Language Models. *International Conference on Learning Representations*, 2020. <https://papers.lunadong.com/paper/4555>
- Kim, H., Kim, Y. J., and Bak, J. *PEMA: An Offsite-Tunable Plug-in External Memory Adaptation for Language Models*, 2023. <https://arxiv.org/abs/2311.08590v3>

- Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. *Large Memory Layers with Product Keys*, 2019. <https://arxiv.org/abs/1907.05242v2>
- Lewis, P., Perez, E., Piktus, A., and others. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172v3>
- Lyu, Q., Shridhar, K., Malaviya, C., Zhang, L., Elazar, Y., Tandon, N., Apidianaki, M., Sachan, M., and Callison-Burch, C. *Calibrating Large Language Models with Sample Consistency*, 2024. <https://arxiv.org/abs/2402.13904v2>
- Miao, M. M., and Ungar, L. *Closing the Confidence-Faithfulness Gap in Large Language Models*, 2026. <https://arxiv.org/abs/2603.25052v2>
- Nelson, E., Kollias, G., Das, P., Chaudhury, S., and Dan, S. *Needle in the Haystack for Memory Based Large Language Models*, 2024. <https://arxiv.org/abs/2407.01437v2>
- OLAResearch. *Cross-Model Memory Transfer via Target-Side Reader Adaptation*, 2026. <https://github.com/OLAResearch/XMemTransfer>
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. *MemGPT: Towards LLMs as Operating Systems*, 2023. <https://arxiv.org/abs/2310.08560v2>
- Pan, X., Yao, W., Zhang, H., Yu, D., Yu, D., and Chen, J. *Knowledge-in-Context: Towards Knowledgeable Semi-Parametric Language Models*, 2022. <https://arxiv.org/abs/2210.16433v3>
- PioneerQyw. *NGM: A Plug-and-Play Training-Free Memory Module for LLMs*, 2026. <https://github.com/PioneerQyw/NGM>
- Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, 2025. <https://arxiv.org/abs/2501.13956v1>
- Robertson, S., and Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009.
- Shin, S., Cha, M., Lee, B.-J., and Park, S. *ERA: Evidence-based Reliability Alignment for Honest Retrieval-Augmented Generation*, 2026. <https://arxiv.org/abs/2604.20854v1>
- Wang, H., Luo, Y., and others. *MemTrapBench: Benchmarking Cognitive Traps in LLM Memory Use*, 2026. <https://www.semanticscholar.org/paper/MemTrapBench%3A-Benchmarking-Cognitive-Traps-in-LLM-Wang-Luo/736d61825a5afed4c85b227951a9880d01e2299f>
- Xu, F. F., Alon, U., and Neubig, G. *Why do Nearest Neighbor Language Models Work?*, 2023. <https://arxiv.org/abs/2301.02828v2>
- Xu, J., Shi, H., Shan, Y., Liu, P., Bai, Y., Li, N., and Liu, X. *TrustMargin: Training-Free Arbitration between Parametric Memory and Retrieved Evidence in Large Language Models*, 2026. <https://arxiv.org/abs/2606.08397v1>
- Yu, Z., Zhao, Y., Cohan, A., and Zhang, X.-P. *HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation*, 2024. <https://arxiv.org/abs/2412.21199v2>
- Yu, Z., Xing, W., Wei, Y., Yang, B., Ye, C., Li, G., and Han, M. *The Attribution Blind Spot: Detecting When Language Models Rely on Memory Rather Than Retrieved Context*, 2026. <https://arxiv.org/abs/2605.26778v1>
- Zhang, Z., Zeng, Z., Lin, Y., Wang, H., Ye, D., Xiao, C., Han, X., Liu, Z., Li, P., Sun, M., and Zhou, J. *Plug-and-Play Knowledge Injection for Pre-trained Language Models*, 2023. <https://arxiv.org/abs/2305.17691v2>
- Zheng, L., Chiang, W.-L., Sheng, Y., and others. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, 2024. <https://arxiv.org/abs/2306.05685>
- Zheng, Y., Wen, B., and Wang, X. *Lngram v2: Latent N-Gram Memory with Interpretable Discrete Representations*, 2026a. <https://arxiv.org/abs/2609.03426v1>
- Zheng, Y., Xia, G., Wang, X., and Ren, L. *Lngram: N-gram Conditional Memory in Latent Space*, 2026b. <https://arxiv.org/abs/2605.24869v1>
- Zhong, Z., Lei, T., and Chen, D. *Training Language Models with Memory Augmentation*, 2022. <https://arxiv.org/abs/2205.12674v3>
- Zhou, W., Gu, Y., Iacovides, G., Qiu, Y., Zhao, Q., and Mandic, D. *Tensorizing Engram: Sharing Latents Across N-Gram Embeddings is Beneficial in LLMs*, 2026. <https://arxiv.org/abs/2606.08347v1>